

SIMULADO — APPLE DEVELOPER ACADEMY UCB

30 questões — 60 pontos — TEMPO MÁXIMO: 3 HORAS

Questão 1 [PESO 1]

Considere a proposição lógica: "Se o código compila, então não há erros de sintaxe". Assinale a alternativa que apresenta a equivalência lógica correta dessa proposição (sua contrapositiva):

- A) Se não há erros de sintaxe, então o código compila.
- B) Se o código não compila, então há erros de sintaxe.
- C) Se há erros de sintaxe, então o código não compila.
- D) O código compila e não há erros de sintaxe.

Questão 2 [PESO 1]

Analise o código em C abaixo e determine a sua saída exata:

```
#include <stdio.h>
int main() {
    int a = 7, b = 2;
    double resultado = a / b;
    printf("%.2f\n", resultado);
    return 0;
}
```

- A) 3.50
- B) 3.00
- C) 3.5
- D) 3

Questão 3 [PESO 1]

Você está desenvolvendo um aplicativo em equipe utilizando o Git. Após realizar diversas alterações no seu código local e testá-las, você deseja registrar essas alterações no histórico do seu repositório local, preparando-as com uma mensagem descritiva, mas ainda sem enviá-las para a nuvem. Qual é a sequência correta de comandos?

- A) git push e git commit
- B) git add e git commit
- C) git commit e git push

D) git add e git push

Questão 4 [PESO 1]

Qual é a saída exata impressa pelo programa abaixo?

```
#include <stdio.h>
int main() {
    int x = 10;
    int y = -x;
    if (y < 0) {
        printf("%d\n", ++x);
    } else {
        printf("%d\n", x++);
    }
    return 0;
}
```

A) -10

B) -11

C) 10

D) 11

Questão 5 [PESO 1]

Na metodologia Challenge Based Learning (CBL), o processo é dividido em três grandes fases que guiam o aprendizado. Assinale a alternativa que apresenta corretamente as três fases fundamentais do framework Apple CBL:

A) Planejar, Executar e Avaliar (Plan, Do, Check).

B) Engajar, Investigar e Agir (Engage, Investigate, Act).

C) Empatizar, Idear e Prototipar (Empathize, Ideate, Prototype).

D) Descobrir, Desenvolver e Entregar (Discover, Develop, Deliver).

Questão 6 [PESO 1]

Dentro do framework Scrum, qual cerimônia tem como objetivo principal promover o alinhamento diário da equipe de desenvolvimento, identificando impedimentos e sincronizando as atividades para as próximas 24 horas?

A) Sprint Planning (Planejamento da Sprint)

B) Sprint Review (Revisão da Sprint)

C) Sprint Retrospective (Retrospectiva da Sprint)

D) Daily Scrum (Reunião Diária)

Questão 7 [PESO 1]

De acordo com as Leis de De Morgan na lógica proposicional, a negação da proposição "O servidor está online e o banco de dados está atualizado" é:

- A) O servidor está offline e o banco de dados está desatualizado.
- B) O servidor não está online ou o banco de dados não está atualizado.
- C) Se o servidor não está online, então o banco de dados não está atualizado.
- D) O servidor não está online e o banco de dados não está atualizado.

Questão 8 [PESO 1]

Análise o programa abaixo. Qual será o valor impresso?

```
#include <stdio.h>
int main() {
    int valores[] = {5, 10, 15, 20};
    int indice = 3;
    printf("%d\n", valores[indice - 1]);
    return 0;
}
```

- A) 10
- B) 15
- C) 20
- D) 5

Questão 9 [PESO 1]

Considere o seguinte algoritmo em pseudocódigo:

```
Início
    A <- 5
    B <- 3
    C <- A + B
    A <- C - B
    B <- C - A
    Imprimir A, B
Fim
```

Quais serão os valores impressos para A e B, respectivamente?

A) 5, 3

B) 8, 3

C) 3, 5

D) 5, 5

Questão 10 [PESO 1]

Qual será a saída impressa pelo seguinte código C?

```
#include <stdio.h>
int main() {
    double valor = 4.567;
    printf("%.1f\n", valor);
    return 0;
}
```

A) 4.5

B) 4.56

C) 4.6

D) 4.57

Questão 11 [PESO 2]

Qual será a saída do programa abaixo?

```
#include <stdio.h>
int main() {
    int a = 9;
    int b = 4;
    double res = (a % b == 0 || !0) ? (double)a / b : (double)(a / b);
    printf("%.2f\n", res);
    return 0;
}
```

A) 2.00

B) 2.25

C) 0.00

D) Erro de compilação

Questão 12 [PESO 2]

Você realizou um commit no Git com as alterações de uma nova funcionalidade, mas percebeu que esqueceu de incluir um arquivo importante que deveria estar nesse mesmo commit. Você deseja desfazer o commit, mas manter as alterações já feitas na sua área de trabalho (working directory) para fazer um novo commit correto. Qual comando resolve esse cenário?

- A) git reset --hard HEAD~1
- B) git revert HEAD
- C) git reset --soft HEAD~1
- D) git checkout HEAD~1

Questão 13 [PESO 2]

Considere as seguintes premissas de um argumento válido:

- I. Se o software não tiver bugs, o cliente ficará satisfeito.
- II. Se o cliente ficar satisfeito, o contrato será renovado.
- III. O contrato não foi renovado.

A partir dessas premissas, é logicamente correto concluir que:

- A) O cliente ficou satisfeito e o software teve bugs.
- B) O software teve bugs.
- C) O cliente não ficou satisfeito, mas o software não teve bugs.
- D) O contrato não foi renovado porque o cliente pediu alteração no software.

Questão 14 [PESO 2]

Qual é a saída do código a seguir?

```
#include <stdio.h>
```

```
int main() {  
    int x = 12, y = 5;  
    int *p = &x;  
    *p = *p + y;  
    p = &y;  
    *p = x - y;  
    printf("%d %d\n", x, y);  
    return 0;  
}
```

A) 17 12

B) 17 5

C) 12 12

D) 12 5

Questão 15 [PESO 2]

No contexto da fase "Investigar" do CBL, a equipe elabora "Guiding Questions" (Questões Guias). O principal propósito das Questões Guias é:

A) Determinar o orçamento necessário para construir a solução do aplicativo final.

B) Identificar o que a equipe precisa saber para compreender melhor o Desafio e embasar a solução futura.

C) Substituir a "Big Idea" (Grande Ideia) caso o desafio se torne muito complexo.

D) Avaliar individualmente a participação técnica de cada membro no repositório do projeto.

Questão 16 [PESO 2]

Durante uma Sprint de 2 semanas, a equipe de desenvolvimento de um app iOS percebe, no 5º dia, que uma das tarefas da Sprint Backlog é tecnicamente inviável no prazo estipulado devido a uma limitação na API de terceiros. Seguindo os princípios ágeis, o que a equipe deve fazer?

A) Continuar trabalhando na tarefa até o final da Sprint, mesmo que ela falhe, e só relatar na Sprint Review.

B) Parar a Sprint imediatamente e iniciar uma nova Sprint Planning.

C) Levar o impedimento à Daily Scrum, discutir o escopo com o Product Owner e adaptar o plano (Sprint Backlog).

D) Alocar horas extras obrigatórias para tentar reescrever a API de terceiros antes do final da Sprint.

Questão 17 [PESO 2]

Qual será o valor da variável **soma** ao final da execução?

```

#include <stdio.h>
int main() {
    int arr[] = {2, 3, 4, 5, 6};
    int soma = 0;
    for (int i = 0; i < 5; i++) {
        if (arr[i] % 2 != 0) {
            continue;
        }
        soma += arr[i];
    }
    printf("%d\n", soma);
    return 0;
}

```

- A) 20
- B) 12
- C) 8
- D) 15

Questão 18 [PESO 2]

Análise o comportamento de uma fila (FIFO) e uma pilha (LIFO). Um algoritmo insere os números 1, 2 e 3 em uma fila vazia. Em seguida, ele remove todos os elementos da fila, um por um, e, a cada remoção, insere o elemento imediatamente em uma pilha vazia. Por fim, ele remove um elemento da pilha e o imprime. Qual número é impresso?

- A) 1
- B) 2
- C) 3
- D) Nenhum, ocorre erro na estrutura.

Questão 19 [PESO 2]

A operação lógica condicional (implicação) $P \rightarrow Q$ (Se P então Q) possui a mesma tabela-verdade de qual das expressões abaixo?

- A) $P \wedge Q$ (P e Q)
- B) $\neg P \vee Q$ (Não P ou Q)
- C) $\neg P \wedge \neg Q$ (Não P e Não Q)
- D) $P \vee \neg Q$ (P ou Não Q)

Questão 20 [PESO 2]

Qual é a saída impressa pelo programa abaixo?

```
#include <stdio.h>

void misterio(int a, int *b) {
    a = a * 2;
    *b = *b + a;
}

int main() {
    int x = 3;
    int y = 4;
    misterio(x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```

A) 6 10

B) 3 10

C) 6 4

D) 3 4

Questão 21 [PESO 3]

Análise o programa abaixo. Qual será o valor exato impresso?

```
#include <stdio.h>

int main() {
    int a = 10;
    int *p1 = &a;
    int **p2 = &p1;

    **p2 = 20;
    (*p1)++;
    a += 5;

    printf("%d\n", **p2);
    return 0;
}
```

A) 10

B) 21

C) 26

D) 16

Questão 22 [PESO 3]

Numa ilha, há apenas dois tipos de habitantes: Cavaleiros (que sempre dizem a verdade) e Valetes (que sempre mentem). Você encontra três habitantes, A, B e C.

A diz: "Todos nós somos Valetes".

B diz: "Exatamente um de nós é Cavaleiro".

O que são A, B e C, respectivamente?

A) Valete, Cavaleiro, Valete.

B) Valete, Valete, Cavaleiro.

C) Cavaleiro, Valete, Cavaleiro.

D) Valete, Cavaleiro, Cavaleiro.

Questão 23 [PESO 3]

Qual será a saída do programa?

```
#include <stdio.h>

void processa(int *v, int tamanho) {
    for (int i = 0; i < tamanho - 1; i++) {
        *(v + i) = *(v + i + 1) - v[i];
    }
}

int main() {
    int vetor[] = {1, 3, 7, 15};
    processa(vetor, 4);
    printf("%d %d\n", vetor[1], vetor[3]);
    return 0;
}
```

A) 4 15

B) 2 8

C) 4 8

D) 2 15

Questão 24 [PESO 3]

Sua equipe está no último dia da Sprint e o objetivo é entregar a integração de uma API de pagamentos. O desenvolvedor responsável terminou o código na branch `feature/pagamentos` e fez um Pull Request para a branch `main`. No entanto, o GitHub acusou um "Merge Conflict" porque outro colega modificou as mesmas linhas na `main` no dia anterior. Considerando práticas ágeis e o uso do Git, a atitude mais adequada é:

- A) Aprovar o Pull Request forçadamente (Force Merge), pois entregar a meta da Sprint no prazo é prioridade absoluta no Scrum.
- B) O desenvolvedor deve realizar um `git checkout main`, um `git pull`, voltar para sua branch `feature/pagamentos`, executar `git merge main`, resolver os conflitos localmente, commitar e atualizar o Pull Request.
- C) Criar uma nova branch a partir da `main`, copiar e colar o código manualmente ignorando a branch conflitante, abrindo mão do histórico de commits.
- D) Cancelar a Sprint atual durante a Daily Scrum, reverter as alterações da `main` com `git reset --hard` e adiar a funcionalidade de pagamentos.

Questão 25 [PESO 3]

Analise cautelosamente o curto-circuito e os operadores no código abaixo. Qual será o valor de `x`, `y` e `z` no final?

```
#include <stdio.h>
int main() {
    int x = 0, y = 3, z = 5;

    if (x++ && --y || ++z % 2 == 0) {
        y += 2;
    } else {
        x += 2;
    }

    printf("%d %d %d\n", x, y, z);
    return 0;
}
```

- A) 1 5 6
- B) 1 3 6
- C) 3 3 6
- D) 1 5 5

Questão 26 [PESO 3]

Analise a função a seguir que manipula uma matriz 3x3 alocada de forma estática.

```

#include <stdio.h>
int main() {
    int mat[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    int *p = &mat[0][0];
    int soma = 0;

    for(int i = 0; i < 9; i += 4) {
        soma += *(p + i);
    }

    printf("%d\n", soma);
    return 0;
}

```

- A) 15
- B) 12
- C) 16
- D) 45

Questão 27 [PESO 3]

Na fase final "Agir" (Act) do Challenge Based Learning, a equipe realiza a "Avaliação" (Evaluation). Em equipes que combinam CBL com Scrum, essa avaliação reflexiva do aprendizado técnico e da colaboração da equipe alinha-se mais perfeitamente a qual evento do Scrum?

- A) Sprint Planning, pois ali a equipe define a solução final a ser codificada.
- B) Product Backlog Refinement, pois prioriza o que deu errado.
- C) Sprint Retrospective, que foca na inspeção do próprio time, processos e ferramentas buscando melhorias.
- D) Daily Scrum, onde o protótipo final é entregue diariamente.

Questão 28 [PESO 3]

Observe a manipulação de tipos e ponteiros no código abaixo:

```
#include <stdio.h>
int main() {
    int a = 17, b = 4;
    double resultado;
    int *p1 = &a;
    int *p2 = &b;

    resultado = (double)(*p1 / *p2) + (*p1 % *p2);

    printf("%.2f\n", resultado);
    return 0;
}
```

- A) 5.25
- B) 5.00
- C) 4.25
- D) 4.00

Questão 29 [PESO 3]

Um algoritmo controla os estados de uma máquina virtual. A máquina inicia no estado 1.

A cada passo, lê um número de uma sequência. Se o número for par, o estado é multiplicado por 2. Se o número for ímpar, soma-se 3 ao estado e em seguida subtrai-se 1.

A sequência de leitura é: 3, 2, 5, 4.

Qual é o estado final da máquina?

- A) 24
- B) 18
- C) 20
- D) 28

Questão 30 [PESO 3]

Qual é a saída gerada pelo seguinte código em C?

```
#include <stdio.h>
int main() {
    int v[] = {10, 20, 30, 40, 50};
    int *p = v;
    int k = 0;

    while(p < v + 5) {
        if (*p % 3 == 0) {
            p += 2;
            continue;
        }
        k += *p;
        p++;
    }
    printf("%d\n", k);
    return 0;
}
```

- A) 60
- B) 120
- C) 30
- D) 70